NewsLens: Text Mining & Document Intelligence on BBC News

Information Retrieval (AIMLCZG537) - Assignment 2 - Group G07 - November 2026

Devansh Sadhotra (2025AG05360) - Gunda Ravindra (2025AG05539) - Vanshika Sangal (2025AG05381) - Chirag Bhatnagar (2025AG05066) - Bhatraju Sreevarsha (2025AG05363)

1. Business context

NewsLens is a document-intelligence tool for the news-monitoring desk of an investment research firm. Analysts receive hundreds of articles a day and need each one routed, grouped, de-duplicated and linked to related coverage before they act on it.

Analyst need	Technique in NewsLens	What the analyst sees
Route each new story to the right desk	Naive Bayes classification	Predicted desk with a confidence score
Spot themes without tagging stories by hand	K-Means clustering	Clusters with their most characteristic terms
Avoid reading the same wire story twice	Exact-hash and cosine duplicate detection	Duplicate groups, flagged or removed
Find related coverage while reading a story	Top-K content-based recommendation	Ranked related articles with scores and shared terms
Understand what the coverage is about	Corpus analytics on TF-IDF	Top terms per desk, term usage by desk

The BBC News corpus stands in for the firm's feed. Its five categories (business, entertainment, politics, sport, tech) play the role of the desks.

2. Dataset

The corpus is the BBC News dataset: 2,224 articles (4.96 MB of text) from 2004-2005, each labelled with one of five categories. We took it from Hugging Face, gopalkalpande/bbc-news-summary (CC0); the original corpus is by Greene and Cunningham (2006).

Property	Value
Documents	2,224 (the Hugging Face copy holds 2,224 of the original 2,225)
Category labels	business 510, sport 510, politics 417, tech 401, entertainment 386
Raw text	4.96 MB, 834,181 words
Words per article	mean 375, median 323, shortest 83, longest 4,374
Exact duplicate pairs	138, none with conflicting labels

Property	Value
Near-duplicate pairs left after exact removal	24 (TF-IDF cosine >= 0.90)

Text and metadata are stored separately and joined on `doc_id`. `scripts/prepare_dataset.py` converts the downloaded CSV into two files:

- `data/bbc_documents.csv`: `doc_id`, `text` (the article body).
- `data/bbc_metadata.csv`: `doc_id`, `title` (headline), `category` (label), `source` ("BBC News"), `summary` (a human-written extractive summary).

IDs such as `BUS-001` come from the category and the article's position. The source has no per-article date, so date is not used. The app loads text and metadata into two separate structures and shows corpus statistics straight after ingestion.

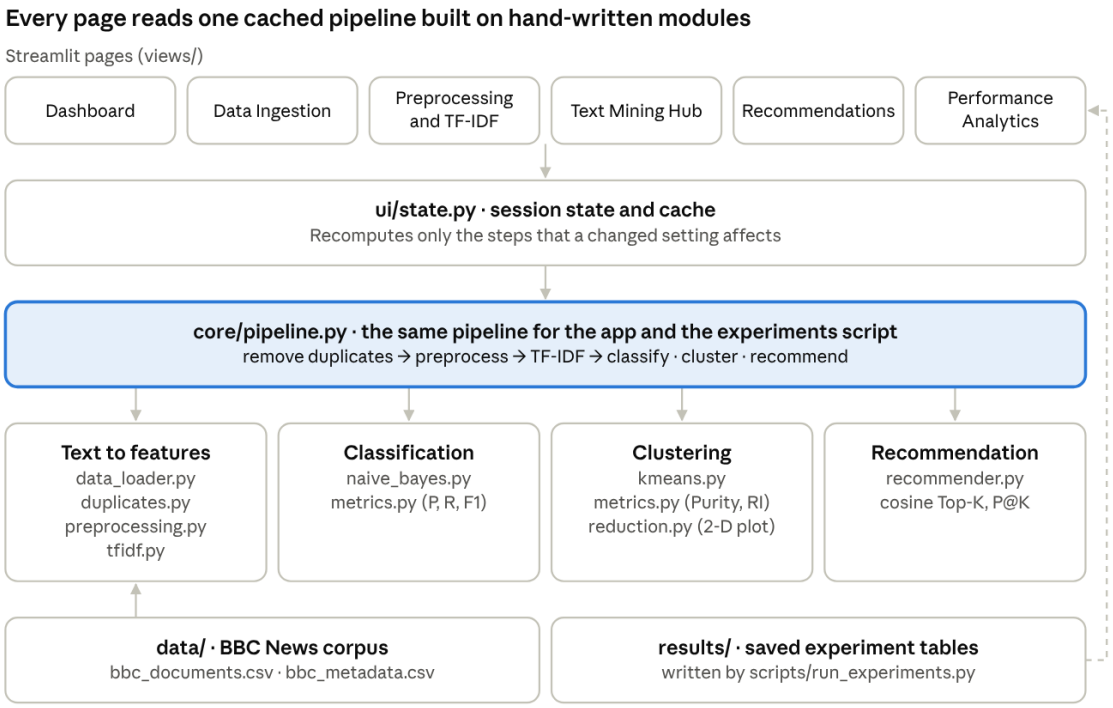
Noise is kept on purpose, so preprocessing and duplicate detection have real work to do:

- Paragraph breaks written as "..": 8,513 occurrences, in every article.
- Inconsistent casing: 810 ALL-CAPS words in 274 articles.
- Numbers and currency: digits in 2,115 articles, 1,392 £ signs (the only non-ASCII character).
- URLs in 3 articles.
- Duplicates: of the 138 duplicate pairs, 133 have identical text (36 of them republished under a different headline) and 5 differ only in casing or punctuation.

The labels support classification and clustering evaluation, and the text supports content-based recommendation. There is no user-interaction data, so collaborative recommendation was not chosen.

3. Implementation approach

NewsLens is a six-page Streamlit app built on a small Python library (`core/`). Every algorithm the brief restricts is written by hand, and the whole workflow runs from the front end.



NewsLens architecture - 6 pages, 1 cached pipeline, 4 groups of core modules

Changing a setting on any page recomputes only the cached steps that depend on it. The experiments script runs the same pipeline, so its saved tables (dashed arrow) match what the app shows.

Page	What the analyst does there
Dashboard	Corpus overview, headline results, links into each workflow step
Data Ingestion	Load the bundled corpus (full or a stratified sample) or upload CSV, JSON or a ZIP of .txt files; map columns; view the text store and metadata table separately; data-quality and duplicate checks
Preprocessing & TF-IDF	Configure the pipeline; step-by-step trace on one article; vocabulary before and after; TF-IDF statistics; inverted-index explorer; cosine-similarity heatmap; duplicate report
Text Mining Hub	Corpus analytics; Naive Bayes training and evaluation; K-Means with its iteration log; Purity, Rand Index and the comparison
Recommendations	Top-K related articles for a chosen article or free text, content-based or hybrid, with scores, metadata and shared terms
Performance Analytics	Headline metrics, runtime per stage, six experiments (run live or loaded from the saved full-corpus run)

Component (written by hand)	File	Notes
Inverted index, TF-IDF, length normalisation, cosine similarity	`core/tfidf.py`	Postings lists in Python dictionaries; numpy only for arithmetic
Multinomial Naive Bayes, stratified train/test split	`core/naive_bayes.py`	Laplace smoothing, computed in log space
K-Means with random or k-means++ initialisation and restarts	`core/kmeans.py`	Records every iteration for the UI
Accuracy, precision, recall, F1, confusion matrix, Purity, Rand Index	`core/metrics.py`	Rand Index by pair counting, checked against a pair-by-pair loop
Exact and near-duplicate detection	`core/duplicates.py`	Normalised-text hash, then a cosine threshold
Top-K recommender and Precision@K	`core/recommender.py`	Content-based, with an optional cluster bonus
PCA for the 2-D plot only	`core/reduction.py`	Randomised SVD in numpy; never used for clustering

Third-party libraries are used only where the brief allows: Streamlit (UI), pandas and numpy (data handling, arithmetic), NLTK (Porter stemmer, WordNet lemmatizer) and Plotly (charts). scikit-learn is not used anywhere.

The hand-written algorithms are checked by 14 unit tests, including two worked examples from Manning, Raghavan and Schütze's *Introduction to Information Retrieval*. Fig. 16.4 gives Purity 0.71 and Rand Index 0.68, and Example 13.1 gives the Naive Bayes class. All seeds are fixed, and `scripts/run_experiments.py` reproduces every result in this report in about one minute on a laptop.

4. Preprocessing and feature engineering

The default pipeline cuts the vocabulary from 70,817 raw word forms to 21,025 terms, and the TF-IDF matrix keeps the 5,000 most widespread. Every step below can be switched on or off on the Preprocessing page, and the app traces each step on any chosen article.

1. Add the headline to the body text.
2. Clean: repair encoding artefacts, strip markup, URLs and e-mail addresses, turn ".." into sentence breaks, collapse whitespace, lowercase.
3. Tokenise with a regular expression; drop numbers, punctuation and words shorter than 2 letters.
4. Remove 164 English stop-words (bundled list). An optional list of 37 news words (said, mr, year...) can be removed too.
5. Normalise words with WordNet lemmatisation (default) or Porter stemming.
6. Optionally add bigrams or trigrams, joined with "_".
7. Remove the 138 exact duplicate copies (default on); near-duplicate removal is optional.

Statistic (2,086 articles)	Before (raw whitespace split)	After preprocessing	Change
Tokens	792,950	444,684	-43.9%
Vocabulary (distinct terms)	70,817	21,025	-70.3%
Terms used only once	40,229	7,349	-81.7%
Tokens per article	380.1	213.2	-43.9%
Share of tokens in the top 20 terms	26.4%	8.9%	-17.5 points

Before preprocessing, the 10 most frequent tokens are all function words (*the, to, of, and, a, in, for, is, that, on*). Afterwards they are news vocabulary (*say, year, mr, make, people, new, one, take, go, game*). Reporting words such as *say* and *mr* survive because they are not standard stop-words; the optional news stop-word list removes them.

Inverted index. Each term maps to its postings list, `{document: term frequency}`, built with plain Python dictionaries; a term's document frequency is the length of its list. For example, *oil* appears in 106 articles (303 times in total). A document-frequency filter then keeps the 12,137 terms that occur in at least 2 articles and at most 90% of them, and the 5,000 with the highest document frequency become the feature vocabulary.

TF-IDF with length normalisation. Weights are computed by hand from the postings lists, with N = 2,086 articles:

w(t, d) = [(1 + log10 tf(t, d)) x log10(N / df(t))] / sqrt(Sum over terms t' in d of [(1 + log10 tf(t', d)) x log10(N / df(t'))]^2)

For *oil* (df = 106, idf = 1.294) in BUS-111 "Indian oil firm eyes Yukos assets", tf = 16 gives a raw weight of 2.852. Dividing by that article's vector length (20.97) gives 0.136. The result is a 2,086 x 5,000 matrix with 260,668 non-zero weights (97.5% sparse), every row of unit length.

Cosine similarity is the dot product of two unit-length vectors:

cos(d1, d2) = dot(d1, d2) / (|d1| x |d2|)

It drives near-duplicate detection, recommendation and the similarity heatmap. On the heatmap, articles from the same category form bright blocks.

5. Classification results (Naive Bayes)

Naive Bayes labels 404 of the 417 held-out articles correctly: 96.9% accuracy, macro F1 0.967. Sport is never missed; the 13 errors are stories that straddle two desks, most often pulled toward tech (5) or between business and politics (4).

Set-up. Multinomial Naive Bayes on term counts, with Laplace smoothing (alpha = 1). The split is a stratified 80/20 (seed 42) that keeps duplicate copies on the same side, giving 1,669 training and 417 test articles, none of which has a copy in training. The document-frequency filter and the 5,000-term feature list are fitted on the training articles only, so test articles never influence which words become features. Class priors from the training set: business 0.240, sport 0.235, politics 0.189, entertainment 0.176, tech 0.161.

Predicted class = argmax over c of [log P(c) + Sum over terms t in d of tf(t, d) x log P(t | c)], where **P(t | c) = (count(t, c) + alpha) / (Sum over t' of count(t', c) + alpha x |V|)**.

Class	Precision	Recall	F1	Test articles
business	0.980	0.960	0.970	100
entertainment	0.986	0.959	0.972	73
politics	0.949	0.949	0.949	79
sport	0.990	1.000	0.995	98
tech	0.929	0.970	0.949	67
Macro average	**0.967**	**0.968**	**0.967**	417
Weighted average	0.969	0.969	0.969	417

Confusion matrix (rows = actual, columns = predicted). The last row compares the predicted class distribution with the actual one.

Actual \ Predicted	business	entertainment	politics	sport	tech	Actual total
business	**96**	0	3	0	1	100
entertainment	0	**70**	1	0	2	73
politics	1	1	**75**	0	2	79

Actual \ Predicted	business	entertainment	politics	sport	tech	Actual total
sport	0	0	0	**98**	0	98
tech	1	0	0	1	**65**	67
Predicted total	98	71	79	99	70	417

Predicted labels: all 13 errors. The app lists all 417 predictions with their confidence and can filter to errors.

Article	Headline	Actual	Predicted	Confidence
POL-046	Drive to 'save' festive holidays	politics	business	0.616
POL-248	Game warnings 'must be clearer'	politics	tech	1.000
POL-253	UK helps raped Rwandan women	politics	entertainment	1.000
POL-391	US casino 'tricks' face ban in UK	politics	tech	0.921
TEC-194	US duo in first spam conviction	tech	business	1.000
TEC-231	World tour for top video gamers	tech	sport	1.000
ENT-115	Wal-Mart is sued over rude lyrics	entertainment	tech	0.505
ENT-171	Row threatens Hendrix museum plan	entertainment	politics	1.000
ENT-332	Glastonbury fans to get ID cards	entertainment	tech	1.000
BUS-005	World leaders gather to face uncertainty	business	politics	0.965
BUS-061	Call to overhaul UK state pension	business	politics	1.000
BUS-070	Pension hitch for long-living men	business	politics	1.000
BUS-340	Making your office work for you	business	tech	1.000

The errors are stories that genuinely straddle two desks: pensions are economic news written in political language, a gaming tour reads like sport, and stories about casino rules, a spam conviction or a lyrics lawsuit borrow another desk's vocabulary. Nine of the 13 errors have a confidence of at least 0.999. Naive Bayes multiplies the evidence of every word as if the words were independent, so its probabilities are overconfident and should not be read as calibrated risk.

Class	Most indicative terms (highest log-probability relative to the other classes)
business	yukos, oil, gm, fiat, shareholder, deutsche, worldcom, ebbers
entertainment	actress, festival, oscar, actor, singer, aviator, film, comedy
politics	tory, labour, lib, kilroy, ukip, chancellor, blair, dems
sport	coach, arsenal, championship, rugby, chelsea, referee, cup, athens

Class	Most indicative terms (highest log-probability relative to the other classes)
tech	user, software, microsoft, gadget, blog, mobile, device, broadband

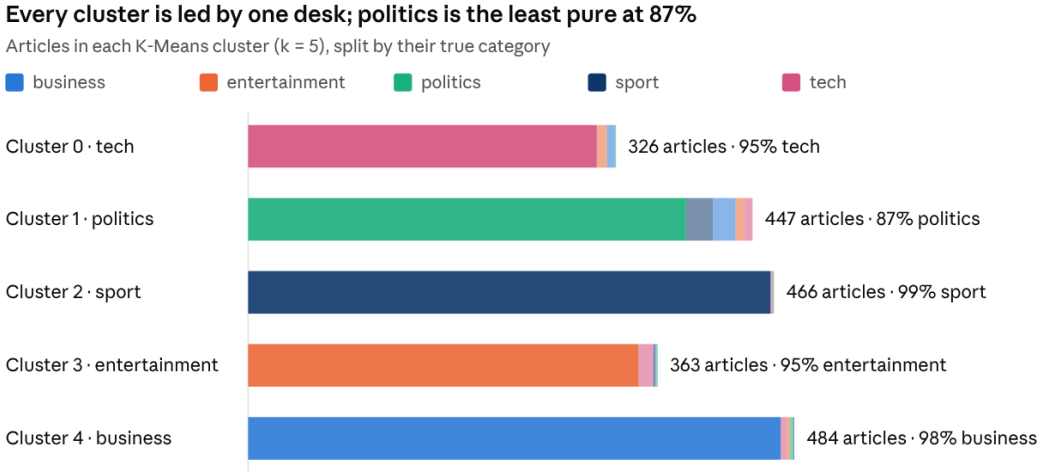
6. Clustering results (K-Means)

K-Means with k = 5 recovers the five desks without seeing a single label: Purity 0.948, Rand Index 0.960, and each cluster lines up with a different category.

Algorithm steps (shown live on the Text Mining Hub page):

1. **Representation:** the 2,086 unit-length TF-IDF vectors. On unit vectors, squared Euclidean distance ranks documents exactly as cosine distance does.
2. **Initialisation:** k-means++, so each new starting centroid is a document drawn with probability proportional to its squared distance from the centroids already chosen. The run is restarted 10 times (seeds 42-51) and the run with the lowest SSE is kept: seed 46, SSE 1,943.74. Its starting documents are TEC-180, POL-163, SPO-184, ENT-095 and BUS-371.
3. **Assignment:** each document joins its nearest centroid.
4. **Update:** each centroid moves to the mean of its documents.
5. **Stopping criterion:** stop when no document changes cluster, when no centroid moves more than 10^-4, or after 100 iterations. The kept run stopped at iteration 6 because no document changed cluster.

Iteration	SSE	Documents reassigned	Largest centroid shift	Cluster sizes (0-4)
1	3,885.26	2,086	0.964	353, 607, 266, 367, 493
2	1,958.53	663	0.107	320, 485, 432, 352, 497
3	1,944.55	88	0.026	324, 457, 461, 357, 487
4	1,943.82	12	0.007	326, 450, 465, 361, 484
5	1,943.75	3	0.004	326, 447, 466, 363, 484
6	1,943.74	0	0.000	326, 447, 466, 363, 484



results/baseline_contingency.csv - default settings, 2,086 articles

Each bar is one cluster, sized by its number of articles; the coloured parts are the articles' true categories. Sport forms the purest cluster (99%), while the politics cluster also holds 24 sport and 20 business articles.

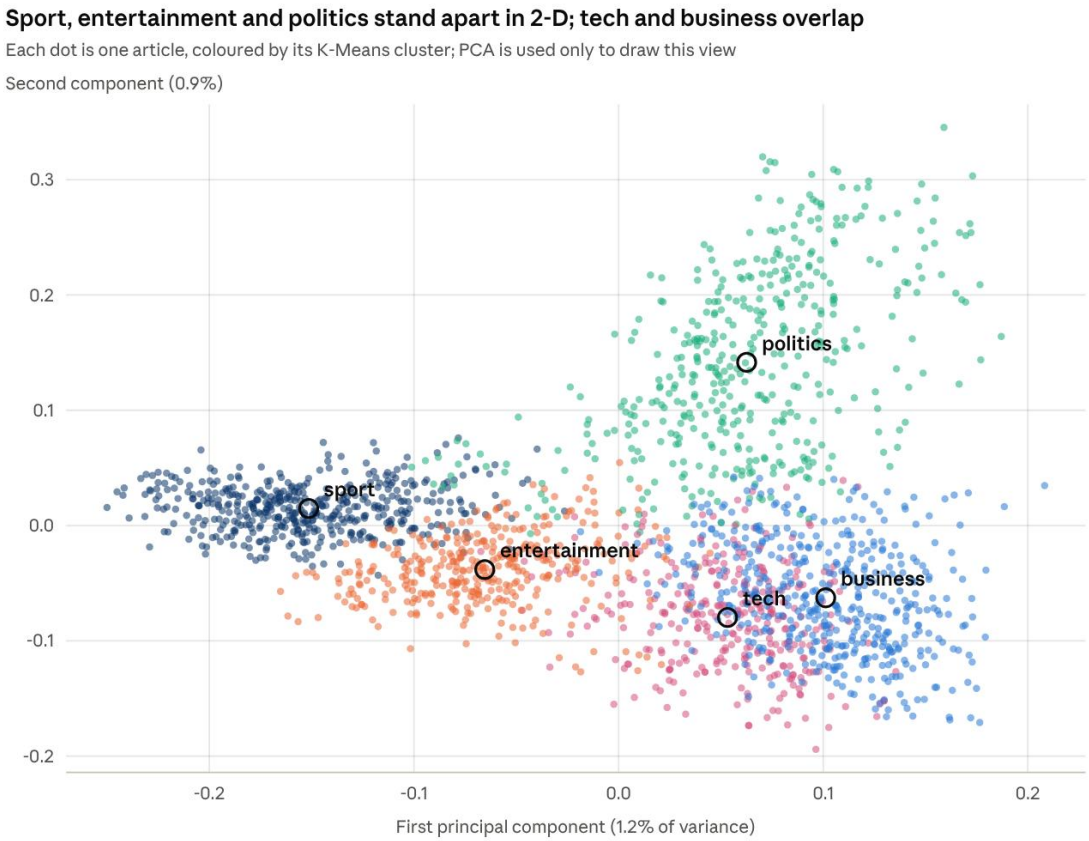
Cluster	Majority category	Top centroid terms
0	tech	technology, user, use, computer, digital, mobile, online, phone
1	politics	labour, election, party, tory, minister, blair, government, mr
2	sport	game, match, win, player, play, cup, team, champion
3	entertainment	film, star, award, actor, best, singer, music, oscar
4	business	bn, market, company, firm, growth, rise, bank, analyst

Purity and Rand Index are computed by our own functions in `core/metrics.py`:

Purity = (1 / N) x Sum over clusters k of max over classes j of |cluster k intersect class j| = (309 + 388 + 463 + 346 + 472) / 2,086 = 0.948

Rand Index = (TP + TN) / (TP + FP + FN + TN) = (401,126 + 1,687,140) / 2,174,655 = 0.960

Of the 2,174,655 document pairs, 42,464 are false positives (same cluster, different category) and 43,925 are false negatives (same category, split across clusters).



2-D PCA projection of the 2,086 TF-IDF vectors - colours = K-Means cluster, labelled at each cluster's centre

The two components explain only 2.1% of the variance of a 5,000-dimensional space, so overlap in this picture does not mean overlap in the clustering: tech and business share this 2-D corner, yet their clusters are 95% and 98% pure in the full space. Clustering always runs on the full TF-IDF vectors.

7. Comparison of classification and clustering

Supervised Naive Bayes beats unsupervised K-Means by about 2 points on every measure the two share (Purity 0.969 against 0.945 on the same test articles). That is a small price for needing no labels at all.

Method	Learns from	Evaluated on	Accuracy	Macro F1	Purity	Rand Index
Naive Bayes	1,669 labelled articles	417 test articles	0.969	0.967	0.969	0.977
K-Means, k = 5	no labels	all 2,086 articles	0.948	-	0.948	0.960
K-Means, k = 5	no labels	the same 417 test articles	-	-	0.945	0.958

K-Means accuracy names each cluster after its majority category, which is only meaningful because k equals the number of categories. Purity and Rand Index for Naive Bayes treat its predicted classes as clusters, so both methods can be scored on the same footing.

- **What each method knows.** Naive Bayes is told the five categories and learns which words signal each one. K-Means sees only the vocabulary, yet its clusters coincide with the desks because BBC categories use very different words.
- **What the metrics measure.** Accuracy and F1 score each document against its label. Purity and Rand Index score how well the grouping as a whole matches the labels, whatever the groups are called.
- **Read Rand Index with care.** 77.6% of all document pairs are true negatives (different category, different cluster). With five classes most pairs are "different", so even a mediocre clustering scores a high Rand Index. Purity and the cluster composition chart are better guides.
- **Where they go wrong.** Naive Bayes confuses business and politics (pensions, economic policy). K-Means' politics cluster is the least pure (86.8%): it also absorbs 24 sport and 20 business articles, nearly all athletics doping hearings and court cases, plus government economic policy such as pensions and EU reform.
- **What each costs.** Naive Bayes needs labelled history and must be retrained when desks change. K-Means needs k and is sensitive to its starting points: single runs range from Purity 0.75 to 0.97, so restarts are essential (section 9, Q5).

8. Recommendation results

For any article, the recommender's top 5 related articles share its category 93.0% of the time, or 96.1% with the cluster-aware hybrid.

Approach: content-based Top-K. The query is either an article's TF-IDF vector or free text run through the same preprocessing pipeline and the corpus IDF. It is scored against every article by cosine similarity.

The query itself and its near-copies (cosine >= 0.98) are skipped, and the K highest scores are returned. The optional hybrid adds a bonus for articles in the query's K-Means cluster: score = (1 - w) x cosine + w x [same cluster], with w = 0.2 by default. For each result the app shows its category, cluster, source, summary and the shared terms that contributed most to the score.

Example 1: query article BUS-001 "US consumer confidence up" (business, cluster 4; top terms: wal, mart, retailer, consumer, confidence). All five results are business articles from the BBC News source.

Rank	Article	Headline	Category	Cluster	Cosine	Shared terms
1	BUS-447	Mixed Christmas for US retailers	business	4	0.287	retailer, wal, mart, festive
2	BUS-081	US retail sales surge in December	business	4	0.261	retailer, consumer, expansion, shop
3	BUS-010	French consumer spending rising	business	4	0.241	consumer, household, confidence, economic
4	BUS-097	Consumer spending lifts US growth	business	4	0.236	consumer, reserve, creation, federal
5	BUS-298	Steady job growth continues in US	business	4	0.233	creation, reserve, consumer, federal

Example 2: free-text query "Oil prices rise as OPEC cuts production and markets worry about economic growth".

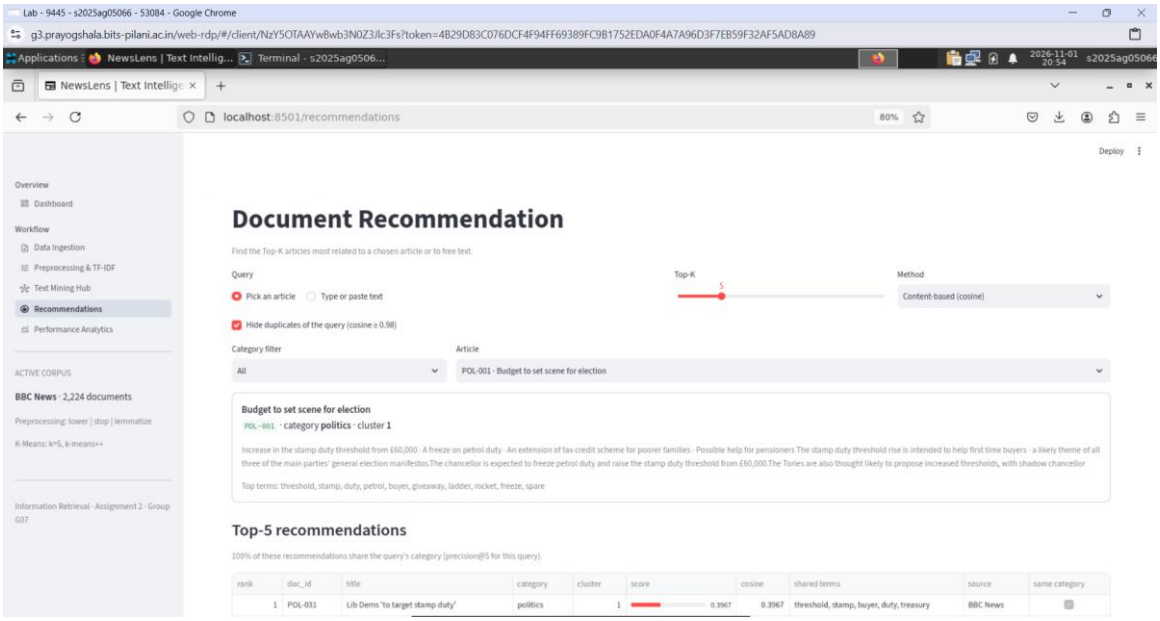
Rank	Article	Headline	Category	Cosine	Shared terms
1	BUS-272	Oil prices fall back from highs	business	0.374	opec, oil, production, price
2	BUS-178	Crude oil prices back above $50	business	0.282	opec, oil, price, production
3	BUS-149	Winter freeze keeps oil above $50	business	0.272	opec, oil, price, production
4	BUS-274	Oil prices reach three-month low	business	0.265	opec, oil, production, price
5	BUS-050	US crude prices surge above $53	business	0.262	opec, production, oil, price

Corpus-wide quality. Every article is used once as the query, and a result counts as relevant when it shares the query's category.

K	Precision@K, content-based	Precision@K, hybrid (w = 0.2)	Precision@K, hybrid (w = 0.5)
1	0.954	0.972	0.970
3	0.940	0.963	0.962
5	0.930	0.961	0.960

K	Precision@K, content-based	Precision@K, hybrid (w = 0.2)	Precision@K, hybrid (w = 0.5)
10	0.919	0.958	0.958

Recommendation interface in the BITS Virtual Lab (1 November 2026). The query is POL-001 "Budget to set scene for election" (politics, cluster 1). All five results share its category; the top one is POL-031 "Lib Dems 'to target stamp duty'" (cosine 0.397, shared terms threshold, stamp, buyer, duty, treasury).



Findings and limitations.

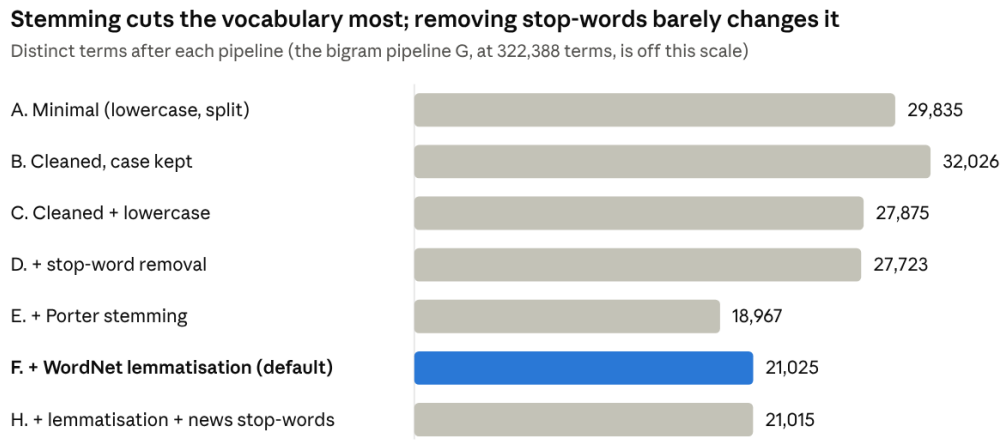
- Precision falls slowly as K grows (0.954 at K = 1 to 0.919 at K = 10): the nearest neighbours are reliable, and the tail drifts toward other desks.
- The cluster bonus helps most at larger K (+3.9 points at K = 10, against +1.9 at K = 1) by pulling in on-topic articles that share fewer exact words. Raising w from 0.2 to 0.5 adds nothing.
- **Vocabulary mismatch:** cosine only matches identical terms, so synonyms ("film" and "movie") and paraphrases are missed.
- **Duplicates crowd the list:** with duplicates kept, 13.9% of top-5 lists contain a copy of the query (section 9, Q2).
- **Shared names mislead:** articles that mention the same person or company can score high while being about different events.
- **No personalisation:** with no reading history, every analyst gets the same list, and the system cannot learn which results were useful.
- **Static index and a coarse proxy:** new articles need the IDF and index rebuilt, and "same category" is only a rough stand-in for "relevant".

9. Inference and discussion

Every answer below rests on the experiments in `results/`, run on the full corpus with fixed seeds. They can be re-run live on the Performance Analytics page.

Q1. How did preprocessing affect the vocabulary, classification, clustering and recommendations?

Preprocessing mostly changes the size and readability of the representation, not its accuracy. The vocabulary shrinks by up to 41% and tokens per article by more than half, while every quality metric stays within about 2 points.



results/exp1_preprocessing_ablation.csv - full vocabulary before the document-frequency filter

Pipeline	Tokens per article	NB accuracy	K-Means Purity	K-Means Rand Index	Recommender P@5
A. Minimal (lowercase, split)	456.8	0.976	0.963	0.972	0.932
B. Cleaned, case kept	369.3	0.974	0.965	0.973	0.932
C. Cleaned + lowercase	369.3	0.971	0.964	0.972	0.931
D. + stop-word removal	213.2	0.962	0.962	0.971	0.933
E. + Porter stemming	213.2	0.974	0.966	0.974	0.933
F. + WordNet lemmatisation (default)	213.2	0.969	0.948	0.960	0.930
G. + lemmatisation + bigrams	425.4	0.969	0.966	0.974	0.939
H. + lemmatisation + news stop-words	195.3	0.969	0.950	0.962	0.931

- **Vocabulary:** lowercasing merges case variants (-13%). Stop-word removal barely changes the vocabulary but removes 42% of the tokens. Stemming shrinks the vocabulary by 32% and lemmatisation by 24%, because stemming is more aggressive. Bigrams multiply it to 322,388 terms, most of them seen once.
- **Classification:** Naive Bayes stays between 96.2% and 97.6% under every pipeline, a spread of six test articles. Removing stop-words costs about one point (97.1% to 96.2%), so for Naive Bayes they carry a little signal rather than pure noise. Adding bigrams leaves accuracy unchanged at 96.9%, because bigrams compete with single words for the 5,000 feature slots.

- **Clustering:** Purity ranges from 0.948 to 0.966. Over 8 further seeds (10 restarts each), lemmatisation and stemming average the same Purity (0.961 and 0.962), so these gaps come from K-Means' starting points, not from the preprocessing.
- **Recommendation:** P@5 moves only between 0.930 and 0.939. Bigrams help most, because two-word phrases make matches more specific.
- **Choice:** we keep lemmatisation as the default. It matches stemming on quality, and the top terms stay readable words (*company*, *tory*) instead of stems such as *compani* and *tori*.

Q2. How do duplicate and near-duplicate documents affect the results, and how were they handled?

Duplicates mainly waste recommendation slots: with them kept, 13.9% of top-5 lists contain a copy of the query, against 0% once exact and near-duplicates are removed.

Scenario	Articles	Test articles with a copy in training	NB accuracy	K-Means Purity	Recommender P@5	Top-5 lists with a copy of the query
1. Keep duplicates, random split	2,224	38	0.966	0.964	0.934	13.9%
2. Keep duplicates, duplicate-aware split	2,224	0	0.969	0.964	0.934	13.9%
3. Remove exact duplicates (default)	2,086	0	0.969	0.948	0.930	1.6%
4. Remove exact and near-duplicates	2,062	0	0.973	0.967	0.929	0.0%

- **Detection:** exact duplicates are found by hashing each article after lowercasing and stripping punctuation, which gives 138 pairs. Near-duplicates are pairs with TF-IDF cosine >= 0.90: 163 with duplicates kept, 24 after exact removal. No pair carries conflicting labels.
- **Classification:** with a random split, 38 test articles have an identical copy in training, so the model is partly tested on text it has already seen. Here the score did not rise, because those stories are easy for the model either way. In general, though, such leakage overstates accuracy, so the duplicate-aware split is the honest estimate.
- **Clustering:** copies count twice in a centroid's mean and pull it toward the duplicated story. The Purity differences between scenarios are of the same size as K-Means' run-to-run variation, so we do not read them as a duplicate effect.
- **Recommendation:** a copy of the query scores a cosine of about 1.0 and takes the top slot while adding nothing new.
- **Handling in NewsLens:** exact duplicates are removed by default, any copies that remain are kept on the same side of the train/test split, near-duplicates can be removed with an adjustable threshold, and the recommender also hides copies of the query (cosine >= 0.98) at query time.

Choosing the near-duplicate threshold. Removing near-duplicates at any cosine threshold from 0.80 to 0.95 keeps clustering and recommendation within half a point, so the result does not hinge on the exact

cut-off. We use 0.90, which removes the 24 clear copies while keeping distinct stories on the same event.

Cosine threshold	Near-duplicates removed	K-Means Purity	K-Means Rand Index	Recommender P@5
0.80	32	0.962	0.972	0.930
0.85	29	0.967	0.974	0.930
0.90	24	0.967	0.975	0.929
0.95	18	0.963	0.972	0.930

Naive Bayes accuracy (0.973 to 0.983) is left out of this table because each threshold leaves a different set of test articles, so the scores are not directly comparable.

Q3. Which recommendation approach was chosen, and how do content-based and collaborative recommendation compare?

We chose content-based recommendation with an optional cluster bonus. The corpus has no reading history, and analysts need related coverage for a story the moment it arrives.

	Content-based (used)	Collaborative filtering
Data required	The documents' text (here TF-IDF vectors)	A user-item interaction matrix (reads, clicks, ratings) with enough overlap between users
New article	Recommendable at once	Cold start until people interact with it
New user	Works without any history	Cold start
What it finds	More of the same topic ("more like this")	What similar users read, including across topics
Explanation	Shared terms	"Readers like you also read..."
Main weaknesses	Over-specialisation, vocabulary mismatch	Sparse data, popularity bias, privacy
Preferable when	Content is fresh and text-rich, users are few (a news desk)	Many users with long histories, items hard to describe from content (music, video), personalisation matters

Once the desk logs which recommendations analysts open, a hybrid of both becomes possible. The cluster bonus is a first step in that direction: it adds a second, content-derived signal to cosine and lifts P@10 from 0.919 to 0.958.

Q4. How do text mining, classification, clustering and recommendation help domain experts decide?

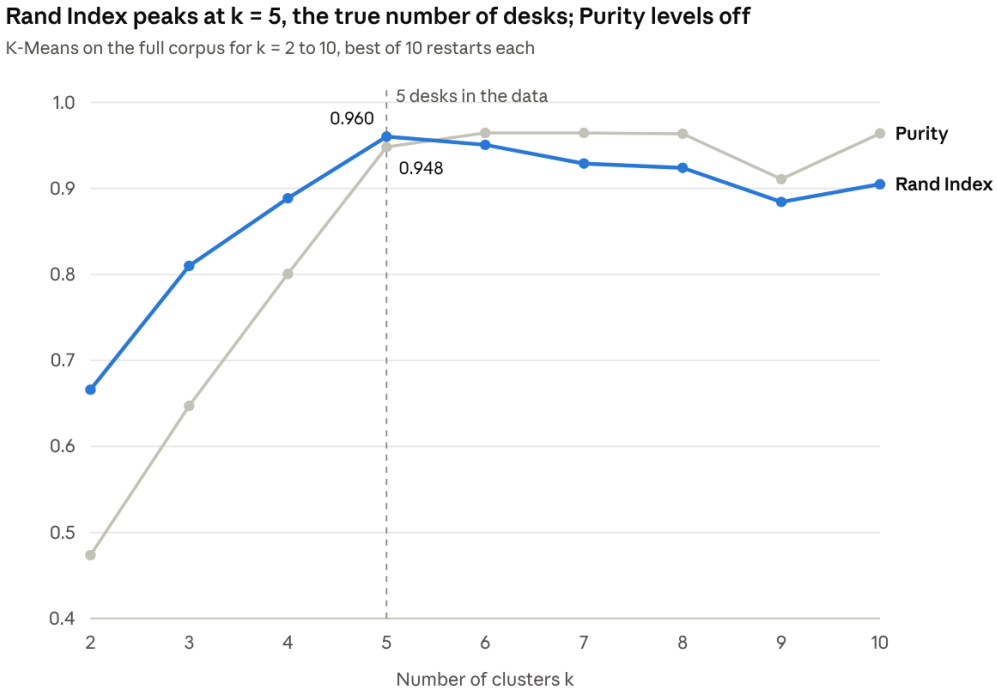
Together they turn a raw feed into de-duplicated, routed, grouped and linked stories, so analysts spend their time on judgement rather than sorting.

- **De-duplication** removes 138 redundant copies, 6.2% of the feed, before anyone reads it.

- **Classification** routes 97% of stories to the right desk automatically. Its confidence is overconfident (9 of 13 errors scored at least 0.999), so a desk should spot-check rather than trust low-scoring routes alone.
- **Clustering** maps the coverage without labels and surfaces cross-cutting themes. The politics cluster, for example, pulled in athletics doping court cases and government economic policy, which a desk-based view would scatter.
- **Recommendation** builds context: reading about US consumer confidence surfaces retail sales, consumer spending and job-growth stories, the inputs to a view on consumer demand.
- **Corpus analytics** (top terms per desk, term usage by desk) shows what each desk is talking about this period.

Q5. How do the Naive Bayes and K-Means results compare, and what are the key findings?

Naive Bayes reaches 96.9% accuracy and macro F1 0.967. K-Means reaches Purity 0.948 and Rand Index 0.960 without labels, about 2 points lower on shared measures (section 7).



results/exp3_k_sweep.csv - k-means++, 10 restarts per k, 2,086 articles

SSE falls steadily from 1,984 at k = 2 to 1,911 at k = 10, so the elbow gives no clear answer; Rand Index does.

Initialisation (k = 5, 10 seeds, one run each)	Purity: worst	Purity: mean	Purity: best	Rand Index: mean
random	0.746	0.818	0.961	0.887
k-means++	0.756	0.836	0.967	0.902

Key findings:

1. Supervision buys about 2 points: Naive Bayes 0.969 against K-Means 0.945 Purity on the same test articles.

2. K-Means is very sensitive to its starting points: a single run can merge two desks and split another (Purity 0.75). k-means++ helps a little, while restarting 10 times and keeping the lowest-SSE run fixes it.
3. The right k shows in Rand Index, not SSE: SSE falls with every extra cluster, Purity levels off near 0.96 for every k above 5, because smaller clusters are easier to keep pure, and Rand Index peaks at k = 5, the true number of desks.
4. Preprocessing changes cost and readability far more than accuracy (Q1), and duplicates matter most for recommendation (Q2).

Limitations and improvements for real-world deployment are set out in section 10.

10. Limitations and improvements for real-world deployment

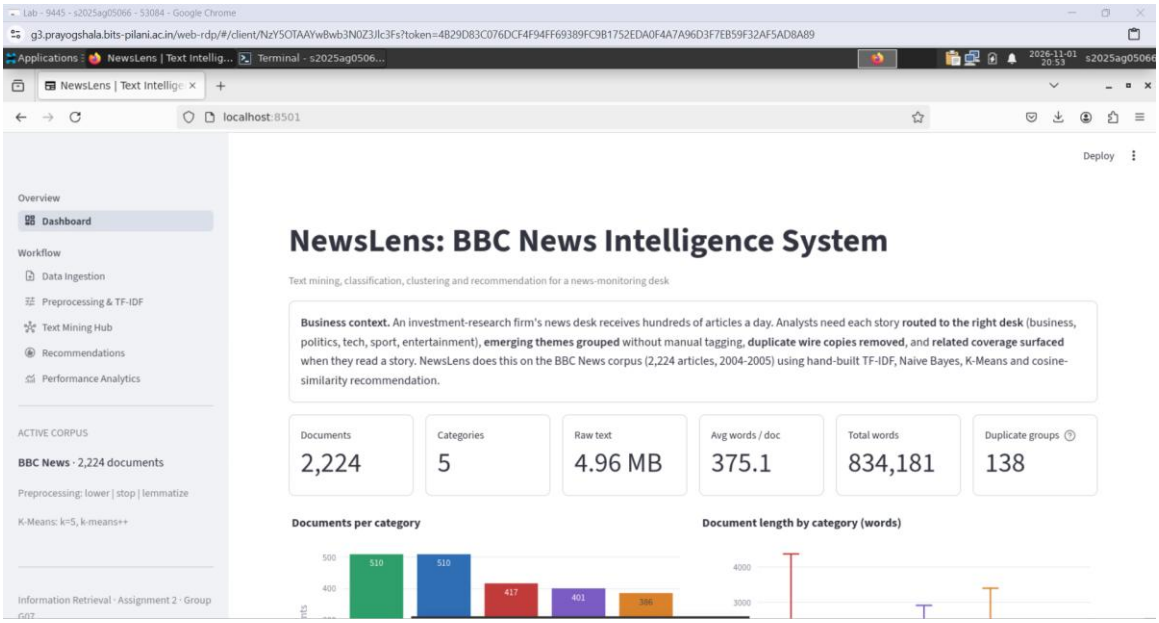
NewsLens works well on a clean archive with five well-separated desks. A live desk would need streaming updates, calibrated confidence and matching on meaning rather than exact words.

Limitation	Effect today	Improvement for deployment
Batch index: IDF and postings are built once	New articles need a full rebuild	Incremental index updates, a scheduled IDF refresh, mini-batch K-Means
Bag-of-words features	Synonyms and paraphrases are missed; word order is lost	Add sentence embeddings or LSA for matching, and keep TF-IDF terms for explanations
Overconfident Naive Bayes probabilities	9 of 13 errors scored at least 0.999	Calibrate the scores or use logistic regression; send low-margin stories to an editor
One label per story	Stories that straddle desks (pensions, sport doping cases) are forced into one	Multi-label classification, so a story can go to two desks
K-Means needs k and is sensitive to its start	Single runs vary from Purity 0.75 to 0.97	Keep restarts, choose k by Rand Index or silhouette, try topic models for evolving themes
Old, clean, single-source corpus (BBC, 2004-2005)	Results may not carry over to noisier, multi-source feeds	Validate on the firm's own feed, monitor drift, retrain on a schedule
No user feedback	Recommendations cannot be personalised or learn from use	Log which recommendations analysts open, then move to a hybrid recommender
Dense matrix and all-pairs similarity	Memory grows with corpus size; duplicate search is quadratic	Sparse matrices, MinHash or LSH for duplicates, approximate nearest-neighbour search
Category used as the relevance proxy	P@K overstates or understates real usefulness	Collect analyst relevance judgements and measure nDCG

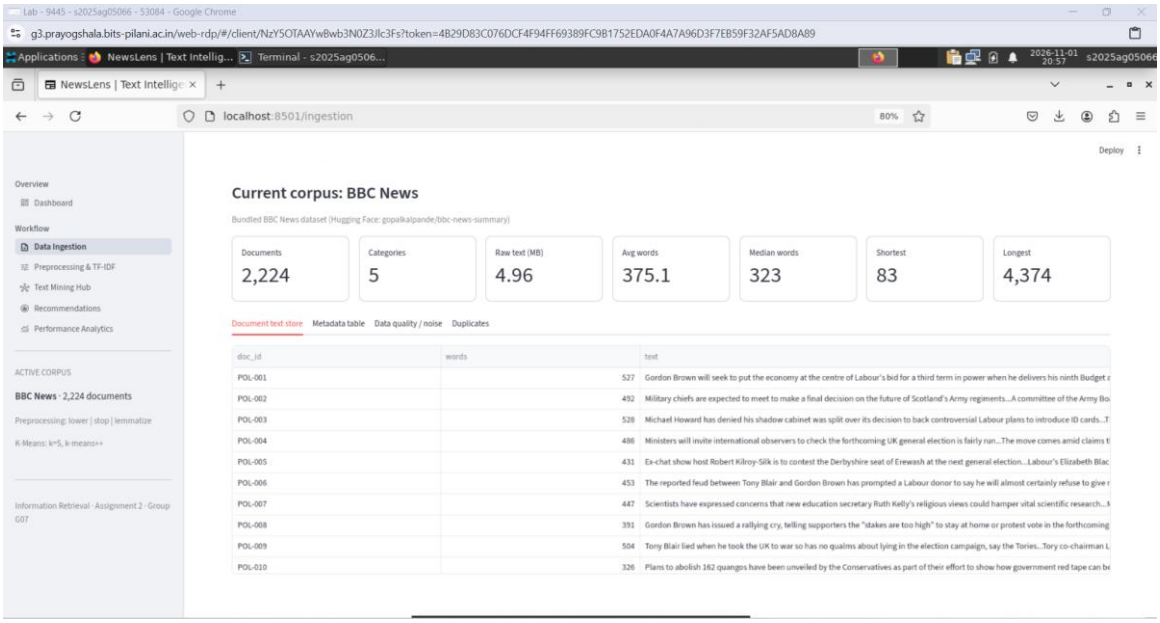
11. Evidence of execution in the BITS Virtual Lab

Lab run, 1 November 2026. NewsLens ran in Firefox on the BITS Virtual Lab desktop (`localhost:8501`) with the full 2,224-article corpus, and every number matched this report. The Recommendations screenshot is in section 8 and the screen recording is in the submission zip.

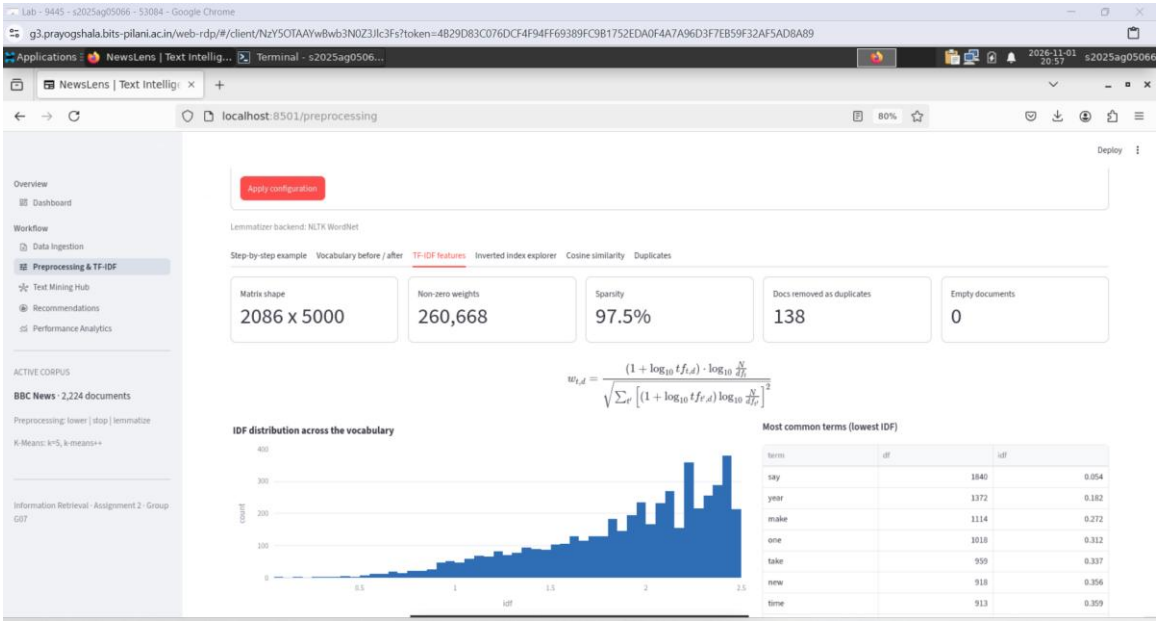
Dashboard. Corpus overview with all 2,224 articles loaded.



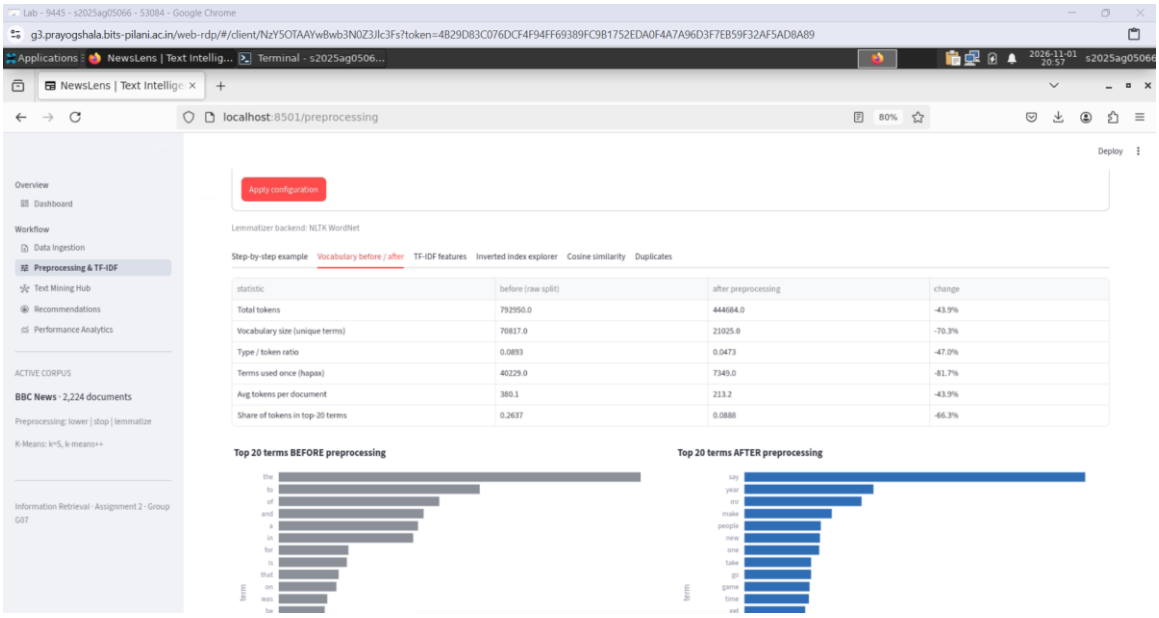
Data Ingestion. Corpus statistics after loading (2,224 documents, 4.96 MB, 5 categories) and the document text store, kept separate from the metadata table.



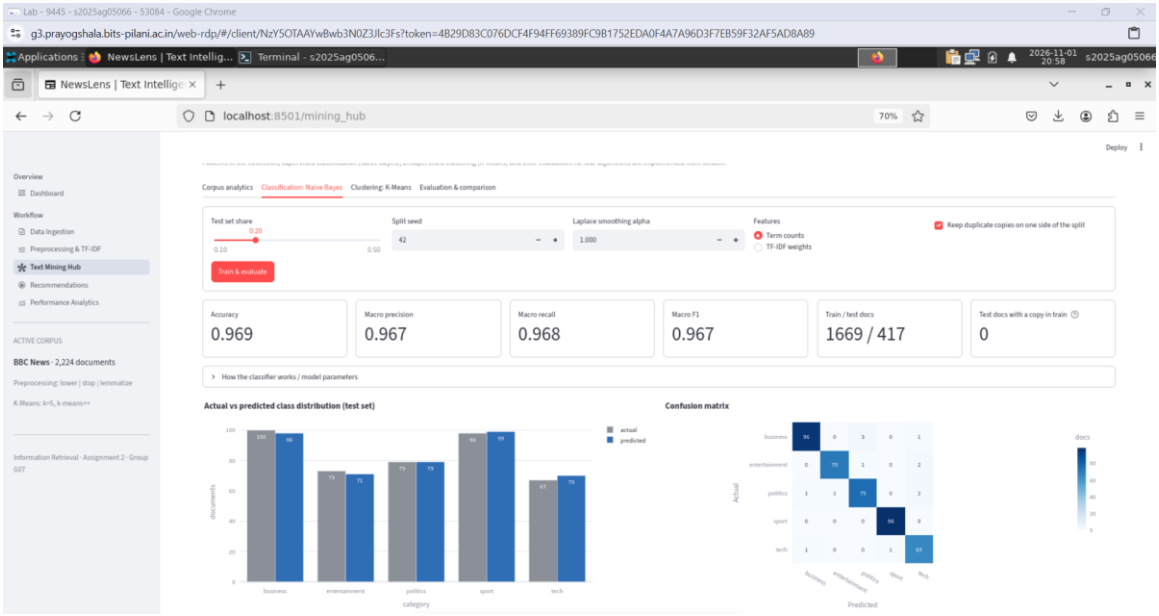
Preprocessing & TF-IDF: TF-IDF features. The lemmatizer backend is NLTK WordNet; the matrix is 2,086 x 5,000 with 260,668 non-zero weights (97.5% sparse) after removing 138 duplicates.



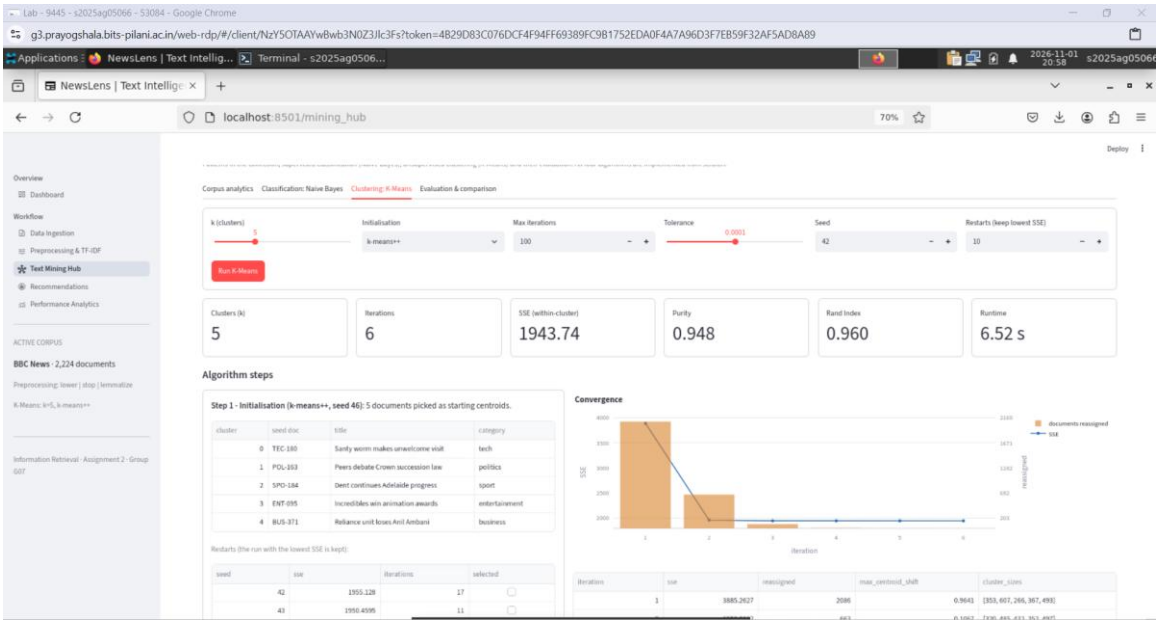
Preprocessing & TF-IDF: vocabulary before and after. Tokens fall by 43.9% and the vocabulary by 70.3%; the top 20 terms change from function words to news vocabulary.



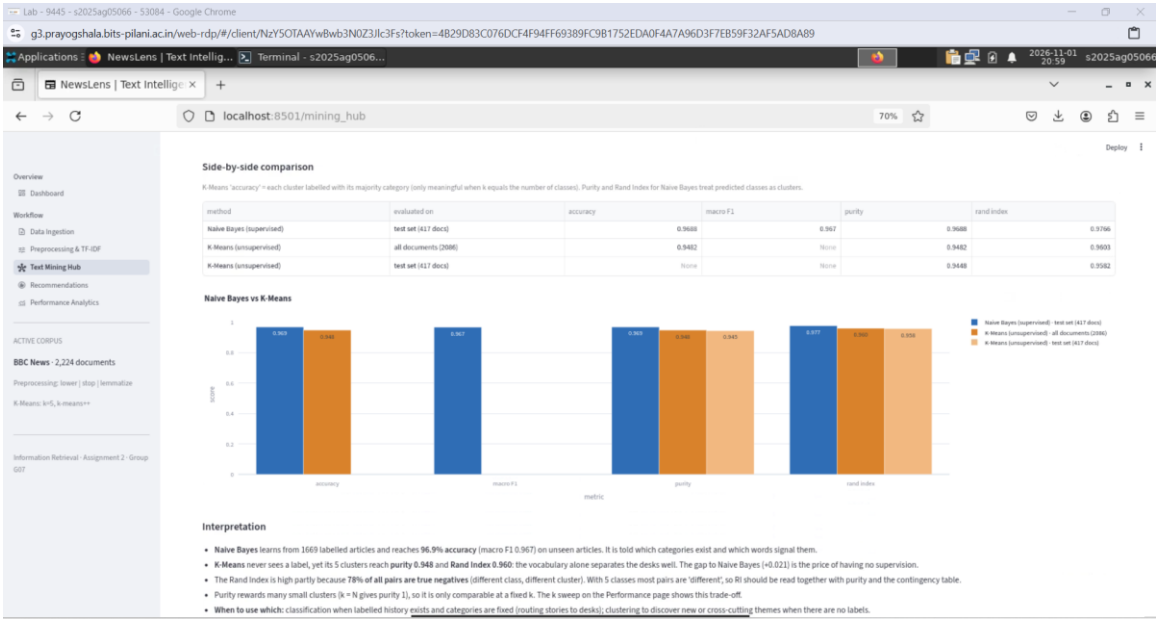
Text Mining Hub: Naive Bayes. Accuracy 0.969 and macro F1 0.967 on 417 test articles, with the actual-versus-predicted chart and the confusion matrix.



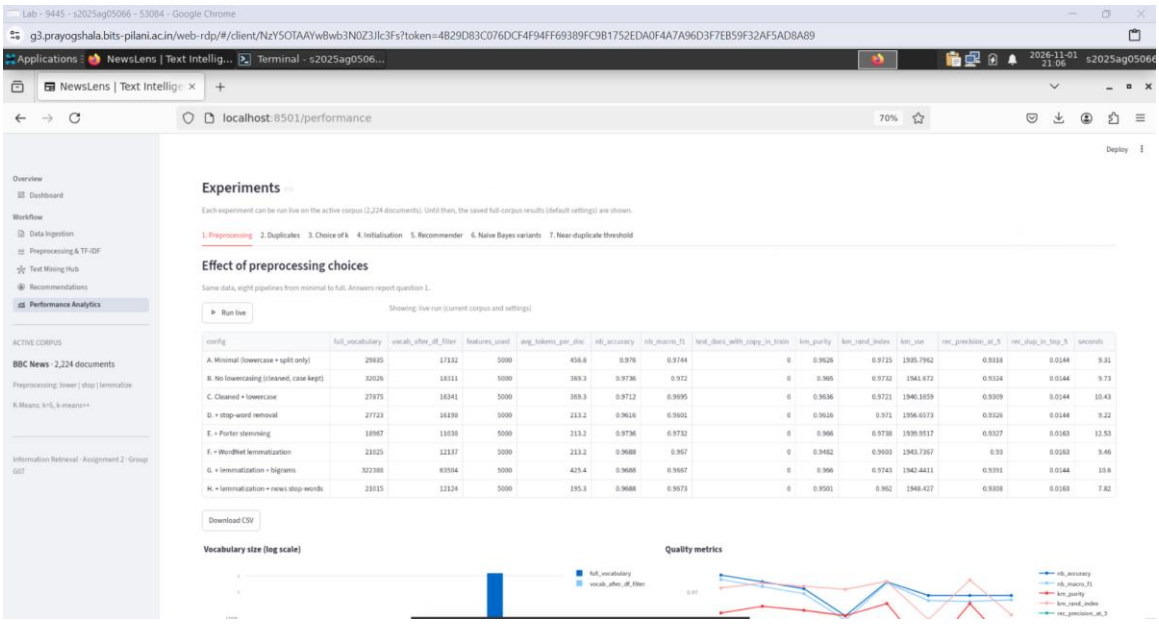
Text Mining Hub: K-Means. k = 5, Purity 0.948, Rand Index 0.960; the kept run (seed 46) converged in 6 iterations, with its starting documents and convergence chart.



Text Mining Hub: evaluation and comparison. Naive Bayes and K-Means side by side on accuracy, macro F1, Purity and Rand Index, with the written interpretation.



Performance Analytics: preprocessing experiment re-run live in the lab. All eight pipelines were re-run on the full corpus inside the lab session ("Showing: live run"). Every value matches the Q1 table in section 9, so the lab reproduces the report's results.



Appendix

Team G07 and contributions (also in `G07_Contribution.xlsx`).

Name	Registration number	Contribution
Devansh Sadhotra	2025AG05360	100%
Gunda Ravindra	2025AG05539	100%

Name	Registration number	Contribution
Vanshika Sangal	2025AG05381	100%
Chirag Bhatnagar	2025AG05066	100%
Bhatraju Sreevarsha	2025AG05363	100%

How to run (full steps in `README.md`):

```
pip install -r requirements.txt
streamlit run app.py
python scripts/run_experiments.py
python -m tests.test_core
```

Submission contents (`G07_Assignment2.zip`): Streamlit source code (`app.py`, `core/`, `ui/`, `views/`, `scripts/`, `tests/`), the dataset (`data/`), saved results (`results/`), this report as PDF, lab screenshots and recording, `README.md`, `requirements.txt` and `G07_Contribution.xlsx`.

Sources

- BBC News Summary dataset, Hugging Face (gopalkalpande/bbc-news-summary)
- D. Greene and P. Cunningham, "Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering", ICML 2006 (original BBC corpus).
- C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008 (TF-IDF, Naive Bayes Example 13.1, Purity and Rand Index Fig. 16.4).